Solution 1. The exercise asks to find, for each shore $R \subseteq V \setminus \{s\}$, a solution (y, z), where $z \geq B_D^{\mathsf{T}} y$ and $y_s - y_r \geq 1$. Also, we have that the inequality $u^{\mathsf{T}} z \leq u(\delta_D^{\text{out}}(R))$ should be true. Given that, I built z such that, for each $a \in A(D)$, we have $z_a = 1$ if $a \in \delta_D^{\text{out}}(R)$ and $z_a = 0$ otherwise. On the other hand, y was built such that, for each $v \in V(D)$, we have $y_v = 0$ if $v \in R$ and $y_v = 1$ otherwise.

First, $y_s - y_r \geq 1$ because $r \in R$ and $s \notin R$, so $y_r = 0$ and $y_s = 1$.

Second, consider an arbitrary arc $a \in A(D)$ from $u \in V(D)$ to $w \in V(D)$. Using the definition of B_D^{T}, we have that $(B_D^{\mathsf{T}} y)_a = \sum_{v \in V}((B_D^{\mathsf{T}})_{a,v})(y_v) = y_w - y_u$. Given that, for $z \geq B_D^{\mathsf{T}} y$, we need that $z_a \geq y_w - y_u$ for every $a \in A(D)$. In this scenario, the expression $y_w - y_u$ is greater than zero only if $y_w = 1$ and $y_u = 0$. In this case, the entry z_a will be equal to 1 because, acording to my formulation, a will be in $\delta_D^{\text{out}}(R)$. As $z_a > 0$ for every a, we can conclude that $z \geq B_D^{\mathsf{T}} y$.

Finally, using the z that I described, we have that $u^{\mathsf{T}} z = \sum_{a \in A(D)} u_a z_a = \sum_{a \in \delta_D^{\text{out}}(R)} u_a$. By definition, we have that $u(\delta_D^{\text{out}}(R)) = \sum_{a \in A(D)} u_a$, thus $u^{\mathsf{T}} z \leq u(\delta_D^{\text{out}}(R))$.

I demonstrated that the pair (y, z) I derived from an arbitrary shore meets all the necessary conditions.

Solution 2. Below is the pseudocode requested. The algorithm uses the funciton MaxFlowInst-from-Transshipment(D, u, b) as we may create an auxiliary digraph $D_{\text{aux}} \coloneqq (V \cup \{r, s\}, A_{\text{aux}}, \varphi_{\text{aux}})$ as follows. Set $W \coloneqq \{v \in V : b_v < 0\}$ and set $Y \coloneqq \{v \in V : b_v > 0\}$. I added new vertices $r, s \notin V$, and set $A_{\text{aux}} \coloneqq A \cup W \cup Y$, where we may assume that $\{A, W, Y\}$ is pairwise disjoint (by possible relabeling common elements), where the incidence function φ_{aux} and arc capacities $u_{\text{aux}} : A_{\text{aux}} \to \mathbb{R}_+ \cup \{\infty\}$ are:

- $\varphi_{\text{aux}}(w) \coloneqq rw$, $u_{\text{aux}}(w) \coloneqq -b_w$, for each $w \in W$
- $\varphi_{\text{aux}}(y) \coloneqq ys$, $u_{\text{aux}}(y) \coloneqq b_y$, for each $y \in Y$
- $\varphi_{\text{aux}}(a) \coloneqq \varphi(a)$, $u_{\text{aux}}(a) \coloneqq u(a)$, for each $a \in A$

It is also important to mention that the MaxFlow-MinCut algorithm returns a pair, and the value of the outcome will not be equal to "unbounded". This is because $b \in \mathbb{R}^V$ and every outward arc of r has its capacity equal to minus a value of b.

Algorithm 1 Transshipment(D, u, b)

```
(D_aux, u_aux, r, s) ≔ MaxFlowInst-from-Transshipment(D, u, b)
(__, (x, R)) ≔ MaxFlow-MinCut(D_aux, u_aux, r, s)
for each a ∈ A(D_aux) do
    v, u ← ends[a]
    if v ∈ {r} then
        if x_a < −b_u then
            return (infeasible, V \ R)
        end if
    else if u ∈ {s} then
        if x_a < b_v then
            return (infeasible, V \ R)
        end if
    end if
end for
x_ans ≔ x↑_{A(D)}
return (feasible, x_ans)
```

First, I will prove that, if $x_{rw} = -b_w$ for every $w \in W$ and $x_{ys} = b_y$ for every $y \in Y$, then x_{ans} is b-Transshipment. This is the case where the algorithm returns (feasible, x_{ans}). For an arbitrary $w \in W$, because of the call to MaxFlow-MinCut, $\text{excess}_x(w) = 0$. When we stop taking into account the arc rw as we do when we use x_{ans}, we have $\text{excess}_{x_{\text{ans}}}(w) = -x_{rw}$. This is because rw is the only arc that we added that is in $\delta^{\text{in}}_{D_{\text{aux}}}(w)$ or $\delta^{\text{out}}_{D_{\text{aux}}}(w)$. If $x_{rw} = -b_w$, then $\text{excess}_{x_{\text{ans}}}(w) = b_w$.

Similarly, for an arbitrary $y \in Y$, because of the call to MaxFlow-MinCut, $\text{excess}_x(y) = 0$. When we stop taking into account the arc ys as we do when we use x_{ans}, we have $\text{excess}_{x_{\text{ans}}}(y) = x_{ys}$. This is because ys is the only arc that we added that is in $\delta^{\text{in}}_{D_{\text{aux}}}(y)$ or $\delta^{\text{in}}_{D_{\text{aux}}}(y)$. If $x_{ys} = b_y$, then $\text{excess}_{x_{\text{ans}}}(y) = b_y$. In the last scenario, for an arbitrary $v \in V \setminus (W \cup Y)$, we know that $b_v = 0$ and $\text{excess}_x(v) = 0$. As we did not add arcs which involves this vertex, $\text{excess}_{x_{\text{ans}}}(v) = 0 = b_v$.

Given that, if $x_{rw} = -b_w$ for every $w \in W$ and $x_{ys} = b_y$ for every $y \in Y$, then, for every $v \in V$, we have that $\text{excess}_{x_{\text{ans}}}(v) = b_v$. Thus x_{ans} is b-Transshipment.

Using the same ideia, it is easy to see that if $x_{rw} \neq -b_w$ for an arbitrary $x \in X$, then $\text{excess}_{x_{\text{ans}}}(w) \neq b_w$. Similarly, if $x_{ys} \neq b_y$, then $\text{excess}_{x_{\text{ans}}}(y) \neq b_y$. Since $x < u_{\text{aux}}$, we only have to check if the value of the entry of x is smaller than the associated entry of b. It is important mentioning again that if $b_v = 0$, then $\text{excess}_{x_{\text{ans}}}(v) = 0 = b_v$. This is the case where the algorithm returns (infeasible, $V \setminus R$).

First, the set $S \coloneqq V \setminus R$ is clearly a subset of V such that $\delta^{\text{in}}_D(S) = \delta^{\text{out}}_D(R)$. Finally, I should prove that $b(S) > u(\delta^{\text{in}}_D(S))$. Let $S' \coloneqq S \cup \{s\}$, we have that $u(\delta^{\text{in}}_D(S)) = u_{\text{aux}}(\delta^{\text{in}}_{D_{\text{aux}}}(S')) - u_{\text{aux}}(\{w \in W : \{w\} \cap S \neq \varnothing\}) - u_{\text{aux}}(\{y \in Y : \{y\} \cap R \neq \varnothing\})$. Expanding the expression, we

have that $u(\delta_D^{\text{in}}(S)) = u_{\text{aux}}(\delta_{D_{\text{aux}}}^{\text{in}}(S')) - (-b(W \cap S)) - b(Y \cap (R \setminus \{r\}))$. Moreover, we have that $u(\delta_D^{\text{in}}(S)) = u_{\text{aux}}(\delta_{D_{\text{aux}}}^{\text{in}}(S')) + b(W \cap S) - (b(Y \cap V) - b(Y \cap S))$. Reorganizing the expression and using that $b(S) = b(W \cap S) + b(Y \cap S)$, we have that $u(\delta_D^{\text{in}}(S)) = u_{\text{aux}}(\delta_{D_{\text{aux}}}^{\text{in}}(S')) + b(S) - b(Y)$. In this scenario, it is straightforward that $u_{\text{aux}}(\delta_{D_{\text{aux}}}^{\text{in}}(S')) < u_{\text{aux}}(\delta_{D_{\text{aux}}}^{\text{out}}(\{r\})) = -b(W) = b(Y)$. By that, we conclude that $u(\delta_D^{\text{in}}(S)) < b(S)$, which is exactly what we want. Thus $V \setminus R$ is a certificate of the infeasibility.

Concluding, I proved that we have a b-transshipment if and only if $x_{rw} = -b_w$ for every $w \in W$ and $x_{ys} = b_y$ for every $y \in Y$. I did that by showing that if $x_{rw} = -b_w$ for every $w \in W$ and $x_{ys} = b_y$ for every $y \in Y$, then we have a b-transshipment and if this is false then we return a certificate that we do not have a b-transshipment.

Solution 3. Below is the pseudocode requested.

Algorithm 2 Circulation(D, l, u)

```
b := −B_D l                                   ▷ B_D is the incidence matrix of D
u' := u − l
(outcome, C) := Transshipment(D, u', b)
if outcome = infeasible then
    return (infeasible, C)
end if
return (feasible, C + l)
```

First, it is easy to see that $\mathbb{1}^\top B_D l = 0$ because, by definition, each entry is a sum of values that will be in the expression of the value of other entry in $B_D l$, but with the oposite sign.

In the first two lines of the code, I am defining the arguments of the Transshipment function. On the third line, we have a call to Transshipment algorithm using D, u' and b. Given that, this algorithm returns C such that $B_D C = b$ and $C < u'$ if the outcome is feasible. In this scenario the Circulation function returns (feasible, $C+l$). This is exactly what we want because $B_D(C + l) = B_D C + B_D l = b + B_D l = 0$. Also, we have that $C < u'$, and given that $C \in \mathbb{R}_+$, then $l < C+l < l+u' = u$. Thus, $C+l$ is a circulation that satisfies all the necessary conditions.

On the other scenario, where the outcome is infeasible, we have that C is a subset of V such that $b(C) > u'(\delta_D^{\text{in}}(C))$. Expanding the expression, we have $b(C) > u(\delta_D^{\text{in}}(C)) - l(\delta_D^{\text{in}}(C))$. Then, we have $b(C)+l(\delta_D^{\text{in}}(C)) > u(\delta_D^{\text{in}}(C))$. Using my definition of b, we have that $-\sum_{v\in C} \text{excess}_l(v)+ l(\delta_D^{\text{in}}(C)) > u(\delta_D^{\text{in}}(C))$. Finally, we have that $l(\delta_D^{\text{out}}(C)) > u(\delta_D^{\text{in}}(C))$ as required.

Concluding, I proved that the Circulation algorithm always return the correct answer by showing that the return is correct in every possible scenario.

Solution 4. Below is the pseudocode requested. The algorithm uses the function Bounded-Transshipment-Inst-from-Bounded-Orientation(G, d) which returns (D, u, b). The digraph $D \coloneqq (V, E, \psi)$ is an arbitrary orientation of G, the arc capacities $u : E \to \mathbb{R}_+ \cup \{\infty\}$ is a constant function where $u_a = 1$ for every $a \in E$. Finally, the vector $b \in \mathbb{R}^V$ is such that $b_v = d_v - |\delta_D^{\text{out}}(v)|$.

Algorithm 3 Bounded-Orientation(G, d)

```
(D, u, b) ≔ Bounded-Transshipment-Inst-from-Bounded-Orientation(G, d)
(outcome, C) ≔ Bounded-Transshipment(D, u, b)
if outcome = infeasible then
    return (infeasible, C)
end if
for each a ∈ A(D) do
    if x_a = 1 then
        v, u ← ends[a]
        ψ(a) ≔ uv
    end if
end for
return (feasible, D)
```

First, the Bounded-Orientation algorithm returns (infeasible, C) if the call to Bounded-Transshipment returns infeasible as the outcome. In this case, I will prove that C is a certificate such that $|E_\varphi[C]| > d(C)$, and therefore a correct answer is returned. It follows from Bounded-Transshipment algorithm that $b(C) + u(\delta_D^{\text{out}}(C)) < 0$. Using the above definitions, we have that $(\sum_{v \in C} d_v - |\delta_D^{\text{out}}(v))| + |\delta_D^{\text{out}}(C)| < 0$. Spliting the somation in two somations, we have that $d(C) - (\sum_{v \in C} |\delta_D^{\text{out}}(v)|) + |\delta_D^{\text{out}}(C)| < 0$. It is easy to see that $\sum_{v \in C} |\delta_D^{\text{out}}(v)| = |\delta_D^{\text{out}}(C)| + |E_\varphi[C]|$. Using that, we have that $d(C) - |\delta_D^{\text{out}}(C)| - |E_\varphi[C]| + |\delta_D^{\text{out}}(C)| < 0$, then $|E_\varphi[C]| > d(C)$. This, with the fact that C is obviously a subset of V, proves that the Bounded-Orientation algorithm returns a correct answer in this case.

Now we will look to the other scenario, when the outcome is feasible. As the incidence function of D changes throughout the algorithm, consider for this proof, that D' is the graph orientation wich is the argument of the Bounded-Transshipment function, and that D is the orientation of G that is returned at the last line of the pseudocode. In this case, we have that $x \coloneqq C$ is such that $B_{D'}x \leq b$, where $B_{D'}$ is the incidence matrix of D'. Expanding the expression, by definition, we have that for each $v \in V$, it holds that $x(\delta_{D'}^{\text{in}}(v)) - x(\delta_{D'}^{\text{out}}(v)) \leq d_v - |\delta_{D'}^{\text{out}}(v)|$. Moreover, we have that $(\sum_{a \in \delta_{D'}^{\text{in}}(v)} x_a) - (\sum_{b \in \delta_{D'}^{\text{out}}(v)} x_b) + |\delta_{D'}^{\text{out}}(v)| \leq d_v$. According to exercise 10, if b is integral and if each finite arc capacity is an integer, then x must be integral, which is our case. As $x \leq u$, an entry of x will be 0 or 1. With that in mind, Bounded-Orientation algorithm flips the orientation of the arcs which have x value equal to 1. Given that, we have that $\sum_{a \in \delta_{D'}^{\text{in}}(v)} x_a$ is exactly the number of inward arcs of v in D' which will become outward arcs of v in D. Similarly, we have that $\sum_{b \in \delta_{D'}^{\text{out}}(v)} x_b$ is the number of outward arcs of v in D' which will become inward arcs of v in D. Now it is easy to see that $(\sum_{a \in \delta_{D'}^{\text{in}}(v)} x_a) - (\sum_{b \in \delta_{D'}^{\text{out}}(v)} x_b) + |\delta_{D'}^{\text{out}}(v)| = |\delta_D^{\text{out}}(v)|$. Thus, we have that $|\delta_D^{\text{out}}(v)| \leq d_v$ for each $v \in V$ in the digraph D returned in this scenario.

I proved that the Bounded-Orientation algorithm gives a correct answer in every possible scenario.